

0.1 Giới thiệu chương và yêu cầu hệ thống

Chương này trình bày quá trình thiết kế, hiện thực và kiểm thử một hệ thống nguyên mẫu theo dõi điện tâm đồ thai nhi (fECG) theo thời gian thực, nhằm đưa mô hình KAN-WNETR đã huấn luyện ở các chương trước từ giai đoạn nghiên cứu sang ứng dụng thực tiễn tại đầu giường bệnh nhân.

Về mặt chức năng, hệ thống cần đáp ứng các yêu cầu sau: (i) thu nhận tín hiệu ECG bụng đơn kênh từ cảm biến phần cứng; (ii) trích xuất tín hiệu fECG theo thời gian thực bằng mô hình học sâu; (iii) ước lượng nhịp tim thai (FHR) và nhịp tim mẹ (MHR); (iv) hiển thị dạng sóng cuộn liên tục trên màn hình đầu giường; (v) đưa ra cảnh báo lâm sàng khi nhịp tim thai vượt ngưỡng bất thường; và (vi) hỗ trợ quản lý đồng thời nhiều bệnh nhân thông qua giao diện web.

Về mặt phi chức năng, hệ thống được thiết kế với các ràng buộc: độ trễ xử lý thấp (khoảng 200 ms mỗi chu kỳ suy luận), khả năng vận hành trên thiết bị biên với tài nguyên hạn chế (Raspberry Pi 5), kích thước mô hình gọn nhẹ sau lượng tử hóa (khoảng 29 MB ở định dạng INT8), và đảm bảo an toàn dữ liệu cơ bản thông qua cơ chế xác thực token thiết bị cùng phiên đăng nhập cho bác sĩ.

Lưu ý: Hệ thống được phát triển trong khuôn khổ nghiên cứu và giáo dục, **không phải thiết bị y tế được chứng nhận** theo các tiêu chuẩn quản lý hiện hành.

0.2 Kiến trúc tổng quan

Hệ thống được thiết kế theo mô hình ba tầng (three-tier), bao gồm: **tầng thiết bị** (Raspberry Pi 5), **tầng máy chủ** (FastAPI trên PC) và **tầng hiển thị** (dashboard web). Sơ đồ kiến trúc tổng quan được minh họa trong Hình 0.1.

Tầng thiết bị chịu trách nhiệm thu nhận tín hiệu ECG thô từ module analog front-end ADS1293 với tần số lấy mẫu 250 Hz, tiền xử lý và suy luận mô hình fECG ngay tại chỗ, đồng thời hiển thị dạng sóng cuộn trên màn hình LCD 3,5 inch (480×320 pixel) thông qua ứng dụng PyQt5. Song song, thiết bị cũng đóng vai trò nguồn phát dữ liệu, truyền các bộ ba (t , raw , $fecg$) lên tầng máy chủ qua kênh WebSocket.

Tầng máy chủ tiếp nhận luồng dữ liệu từ thiết bị, thực hiện phân tích lâm sàng (ước lượng FHR, MHR, chỉ số chất lượng tín hiệu, phát hiện cảnh báo), lưu trữ lịch sử mẫu vào cơ sở dữ liệu SQLite, và phân phối (fan-out) dữ liệu cùng các chỉ số phân tích tới tất cả các dashboard web đang đăng ký theo dõi.

Tầng hiển thị là một ứng dụng web được thiết kế theo triết lý “480×320-first” và “touch-first” — nghĩa là giao diện tối ưu cho màn hình cảm ứng nhỏ của Pi, nhưng

vẫn co giãn tốt khi mở trên trình duyệt máy tính hoặc thiết bị có kích thước lớn hơn trong cùng mạng LAN.

Luồng dữ liệu chính của hệ thống diễn ra tuần tự như sau:

ADS1293 (250 Hz) → acquisition → infer.py (trích fECG) → uploader (WebSocket) → server (phân tích FHR/MHR + lưu trữ) → fan-out WebSocket → dashboard web

Điểm đáng lưu ý là module monitor trên thiết bị tái sử dụng chính xác cùng phép toán suy luận của mô hình thông qua `model_loader.py` (import trực tiếp `infer.py`): bao gồm bộ lọc thông dải 3–90 Hz, chuẩn hoá z-score theo từng cửa sổ, phiên ONNX INT8, và lấy kênh fECG = channel index 1 từ đầu ra dual-branch. Không có bất kỳ sửa đổi nào đối với `infer.py` hay thư mục `models/` khi tích hợp vào hệ thống thời gian thực.

Hình 0.1: Kiến trúc tổng quan hệ thống theo dõi fECG thời gian thực (3 tầng: thiết bị Pi 5 – server PC – dashboard web).

0.3 Tối ưu mô hình cho thiết bị biên

0.3.1 Xuất ONNX

Để triển khai mô hình học sâu trên thiết bị biên có tài nguyên hạn chế, bước đầu tiên là chuyển đổi mô hình từ framework PyTorch sang định dạng Open Neural Network Exchange (ONNX). Việc sử dụng ONNX mang lại lợi thế quan trọng: mô hình có thể được suy luận bằng ONNX Runtime — một engine tối ưu hỗ trợ nhiều nền tảng phần cứng mà không phụ thuộc vào PyTorch — từ đó giảm đáng kể dung lượng bộ nhớ và các thư viện phụ thuộc trên Pi.

Mô hình được xuất với kích thước đầu vào cố định $[B, 1, 992]$, tương ứng một cửa sổ tín hiệu đơn kênh dài 992 mẫu (xấp xỉ 3,97 giây ở tần số 250 Hz). Đầu ra có dạng $[B, 2, 992]$ theo kiến trúc dual-branch, trong đó tín hiệu fECG được trích xuất từ kênh chỉ số 1 (`output[0, 1, :]`).

0.3.2 Lượng tử hóa INT8 động (MatMul/Gemm)

Sau khi xuất ONNX, mô hình được tiếp tục tối ưu bằng kỹ thuật lượng tử hóa động INT8 (dynamic quantization). Kỹ thuật này áp dụng biểu diễn số nguyên 8-bit cho các phép toán nhân ma trận (MatMul) và nhân ma trận tổng quát (Gemm) — hai loại phép toán chiếm phần lớn chi phí tính toán trong mạng Transformer. Nhờ đó, kích thước mô hình giảm còn khoảng 29 MB so với phiên bản FP32 gốc, đồng

thời tốc độ suy luận trên CPU ARM của Pi 5 được cải thiện đáng kể nhờ các phép toán số nguyên nhanh hơn phép toán dấu phẩy động.

Đánh đổi chính của lượng tử hóa là sự suy giảm nhỏ về chất lượng tín hiệu đầu ra. Việc so sánh định lượng giữa phiên bản INT8 và FP32 thông qua các chỉ số SSIM và PSNR đã được trình bày chi tiết tại Chương ??.

0.3.3 ONNX Runtime trên Pi 5 và benchmark

Trên Raspberry Pi 5, phiên ONNX Runtime được khởi tạo với provider `CPUExecutionProvider` và số luồng nội bộ (`intra_op_num_threads`) bằng 4, tận dụng toàn bộ bốn lõi CPU Cortex-A76 của bo mạch.

Kết quả benchmark trên Pi 5 cho thấy mỗi cửa sổ 992 mẫu (3,97 giây tín hiệu) được suy luận trong khoảng 80–120 ms, tương đương hệ số thời gian thực (real-time factor) từ 8 đến 12 lần — nghĩa là mô hình xử lý nhanh hơn gấp 8–12 lần so với tốc độ thu nhận tín hiệu. Kịch bản benchmark (`benchmark.py`) đồng thời đo cả chất lượng trích xuất (SSIM, PSNR) và độ trễ trên toàn bộ tập kiểm thử. Bảng 1 tổng hợp các chỉ số chính.

Bảng 1: Benchmark mô hình INT8 trên Raspberry Pi 5.

Chỉ số	Giá trị
Kích thước mô hình (INT8)	29 MB
Latency / cửa sổ (992 mẫu)	~80–120 ms
Hệ số thời gian thực	~8–12×
Val SSIM	~0.796
Val PSNR	~26.82 dB

0.4 Phân hệ thu nhận và thiết bị (Raspberry Pi 5)

0.4.1 Phần cứng

Thiết bị đầu giường được xây dựng trên nền tảng Raspberry Pi 5 phiên bản 4 GB RAM, chạy hệ điều hành Raspberry Pi OS 64-bit (Bookworm) với Python 3.10 trở lên. Các thành phần phần cứng bổ sung bao gồm: module thu nhận tín hiệu ECG tương tự ADS1293 (24-bit, hỗ trợ đa kênh); màn hình LCD cảm ứng 3,5 inch có độ phân giải 480×320 pixel; và nguồn cấp/battery HAT.

Nhằm tối ưu hóa cho môi trường sử dụng thực tế tại đầu giường bệnh nhân, toàn bộ các thành phần trên được lắp ráp và đóng gói gọn gàng bên trong một hộp bảo vệ (case). Hộp đựng được thiết kế vừa khít để cố định chắc chắn màn hình LCD cùng các module mạch điện, đảm bảo tính cách điện, an toàn và chống va đập. Đồng thời, cấu trúc hộp cũng bố trí các khe thông gió phù hợp để hỗ trợ tản nhiệt

hiệu quả, giữ cho Raspberry Pi 5 hoạt động ổn định khi phải chạy thuật toán suy luận học sâu liên tục trong thời gian dài.

0.4.2 Driver ADS1293

Một thách thức kỹ thuật đặc thù khi tích hợp phần cứng là màn hình LCD 3,5 inch sử dụng toàn bộ 26 chân đầu tiên của header GPIO (bao gồm bus SPI0 phần cứng, chân vật lý 1–26). Do đó, module ADS1293 không thể sử dụng SPI phần cứng mà phải được giao tiếp thông qua bus **SPI phần mềm** (bit-banged SPI) trên các chân ở phần trên của header (chân vật lý ≥ 27).

Driver được viết bằng Python sử dụng thư viện `lgpio` — thư viện GPIO cấp thấp chính thức trên Pi 5, thay thế cho `RPI.GPIO` vốn không tương thích với kiến trúc mới. Bảng 2 trình bày sơ đồ đấu nối chi tiết.

Bảng 2: Bảng đấu nối ADS1293 ↔ Raspberry Pi 5 (SPI mềm, chân phía trên).

ADS1293	Tín hiệu	BCM GPIO	Chân vật lý
SCLK	clock	GPIO21	40
SDI	MOSI	GPIO20	38
SDO	MISO	GPIO19	35
CSB	chip select	GPIO26	37
DRDYB	data ready (active-low)	GPIO16	36
VDD	3V3	—	17
GND	ground	—	39

ADS1293 được cấu hình ở chế độ 3-lead với cách đấu nối đầu vào: kênh CH1 nối IN1(+)/IN2(–), kênh CH2 nối IN3(+)/IN4(–), kênh CH3 nối IN5(+)/IN6(–). Mạch right-leg drive (RLD) để giảm nhiễu chế độ chung được kết nối trên chân IN4. Bộ ADC sử dụng dao động nội (internal oscillator) với độ phân giải cao. Giá trị đầu ra 24-bit có dấu được chuyển đổi sang đơn vị mV theo công thức:

$$V_{\text{mV}} = \frac{\text{counts}}{2^{23}} \times V_{\text{ref}} \quad (1)$$

trong đó $V_{\text{ref}} = 2400 \text{ mV}$ là điện áp tham chiếu. Mô hình AI chỉ sử dụng kênh bụng (channel 0), tuy nhiên driver vẫn đọc đầy đủ cả ba kênh nếu các đầu vào được đấu nối.

0.4.3 Xử lý thời gian thực

Mô hình hoạt động theo kiểu batch: mỗi lần suy luận cần một cửa sổ đầy đủ 992 mẫu. Để chuyển đổi sang chế độ hoạt động liên tục theo thời gian thực, hệ thống sử dụng lớp `FecgProcessor` với chiến lược **bộ đệm vòng kết hợp phát lại** (ring

buffer with replay).

Cụ thể, một bộ đệm vòng có kích thước 992 mẫu được duy trì liên tục. Mỗi khi có đủ $HOP = 50$ mẫu mới (tương ứng 200 ms ở 250 Hz, tức chu kỳ tái suy luận khoảng 5 Hz), toàn bộ cửa sổ 992 mẫu được đưa vào mô hình. Sau mỗi lần suy luận, hệ thống “phát lại” 50 mẫu fECG mới nhất từ đầu ra của mô hình, mỗi mẫu thô đầu vào tương ứng với một mẫu fECG đầu ra. Nhờ cơ chế này, đường fECG trên màn hình cuộn mượt mà và giữ đồng bộ với đường tín hiệu thô, với độ trễ xấp xỉ bằng một chu kỳ hop (khoảng 200 ms).

Giao diện sử dụng đơn giản: `proc = FecgProcessor(); fe = proc.push(raw_s` trong đó mỗi lệnh `push()` nhận vào một mẫu thô và trả về mẫu fECG đã căn chỉnh thời gian tương ứng.

0.4.4 Giao diện monitor đầu giường

Ứng dụng monitor được xây dựng bằng PyQt5 kết hợp pyqtgraph, hiển thị toàn màn hình trên LCD 480×320 với nền đen kiểu máy theo dõi y tế. Giao diện gồm hai đường tín hiệu cuộn theo thời gian: phía trên là tín hiệu bụng thô (raw abdominal signal, màu xanh cyan) và phía dưới là tín hiệu fECG đã được mô hình AI trích xuất (màu hồng). Tốc độ làm mới đồ thị là khoảng 30 khung hình/giây (timer 33 ms). Thanh tiêu đề hiển thị mã bệnh nhân, nguồn tín hiệu (ADS1293 hoặc simulator) và trạng thái mô hình. Người dùng nhấn phím `Esc` để thoát.

Bộ thu thập tín hiệu (`Acquirer`) chạy trong luồng nền (background thread), liên tục đọc mẫu từ nguồn tín hiệu, đẩy qua `FecgProcessor` để suy luận, lưu vào bộ đệm vòng chia sẻ, và đồng thời gửi dữ liệu lên server thông qua `Uploader`. Luồng GUI chính chỉ việc đọc snapshot từ bộ đệm chia sẻ trên mỗi chu kỳ timer và cập nhật đồ thị, đảm bảo giao diện luôn phản hồi mượt mà.

0.4.5 Cơ chế dự phòng

Hệ thống được thiết kế với nhiều tầng dự phòng nhằm đảm bảo khả năng phát triển và kiểm thử linh hoạt. Nếu phần cứng ADS1293 hoặc thư viện `lgpio` không khả dụng (ví dụ khi chạy trên máy tính phát triển), ứng dụng tự động chuyển sang chế độ **mô phỏng** (`SimulatedSource`), tạo ra tín hiệu ECG tổng hợp gồm QRS của mẹ (75 bpm, biên độ 1,0) cộng QRS thai nhi (140 bpm, biên độ 0,18) cùng dao động đường cơ sở và nhiễu Gaussian. Ngoài ra, nếu `onnxruntime` hoặc `scipy` không được cài đặt, hệ thống sử dụng bộ lọc placeholder thay thế cho mô hình để giao diện vẫn hoạt động bình thường.

Một công cụ bổ sung là `stream_sim.py`: kịch bản này chạy headless (không

cần GUI hay phần cứng) và stream dữ liệu mô phỏng tới server qua WebSocket y hệt như một thiết bị thật, cho phép kiểm thử toàn bộ chuỗi xử lý từ server đến dashboard mà không cần Pi vật lý.

0.5 Phân hệ server và phân tích lâm sàng (FastAPI + SQLite)

0.5.1 Giao tiếp WebSocket và xác thực token thiết bị

Tầng máy chủ được xây dựng trên framework FastAPI (Python), sử dụng hai kênh WebSocket chính:

Kênh `/ws/device` dành cho thiết bị nạp dữ liệu. Quy trình bắt tay (handshake) diễn ra như sau: thiết bị gửi thông điệp `hello` chứa `patient_id`, `token` và `sample_rate`. Server kiểm tra token với giá trị bí mật `DEVICE_TOKEN` — nếu sai, kết nối bị đóng ngay lập tức; nếu đúng, server trả về thông điệp `ack` kèm `session_id` và bắt đầu nhận các batch mẫu dạng `samples {data: [[t, raw, fecg], ...]}`.

Kênh `/ws/live/{patient_id}` dành cho dashboard đăng ký theo dõi một bệnh nhân cụ thể. Mỗi khi có batch mới từ thiết bị, server phân phối (fan-out) tới tất cả các WebSocket dashboard đang đăng ký, kèm theo các chỉ số phân tích mới nhất `{fhr, mhr, sq, alarm, label}`. Khi dashboard vừa kết nối, server gửi ngay metrics gần nhất để tránh giao diện bị trống.

Về bảo mật, đối với môi trường triển khai thực tế, cần thay đổi giá trị mặc định của `DEVICE_TOKEN` / `FECG_DEVICE_TOKEN` và đặt server sau proxy `HTTPS/wss`.

0.5.2 Ước lượng FHR / MHR / chỉ số chất lượng tín hiệu

Các chỉ số lâm sàng được tính toán hoàn toàn **phía server** (server-side) — do server nhận mọi mẫu từ thiết bị nên đóng vai trò nguồn sự thật duy nhất (single source of truth), tránh việc lặp lại phép toán trên trình duyệt. Mỗi bệnh nhân đang phát dữ liệu được gán một thể hiện `PatientAnalyzer` riêng biệt.

Nhịp tim thai (FHR) được ước lượng từ kênh `fecg` (tín hiệu đã qua mô hình AI), còn **nhịp tim mẹ (MHR)** được ước lượng từ phức bộ QRS trội trong tín hiệu `raw`. Thuật toán phát hiện QRS là một pipeline kiểu Pan–Tompkins được hiện thực thuần bằng NumPy (không phụ thuộc thư viện chuyên dụng), bao gồm các bước:

1. Khử đường cơ sở (baseline removal) bằng trung bình trượt 0,2 giây.
2. Tính đạo hàm (derivative) để nhấn mạnh sườn dốc của phức bộ QRS.

3. Bình phương (squaring) để khuếch đại thành phần tần số cao.
4. Tích phân cửa sổ trượt (moving-window integration) khoảng 40 ms.
5. Xác định ngưỡng thích nghi dựa trên phân vị 99 của tín hiệu đã tích phân, kết hợp thời gian trơ sinh lý (physiologic refractory period) — khoảng 250 ms cho fECG và 350 ms cho mECG — để loại bỏ các đỉnh giả.

Cửa sổ phân tích cuộn có độ dài 6 giây, yêu cầu tối thiểu 2,5 giây tín hiệu trước khi bắt đầu ước lượng. Nhịp tim báo cáo là giá trị BPM tính từ trung vị (median) của các khoảng R–R hợp lệ, được làm trơn bằng bộ lọc trung bình mũ (EMA, $\alpha = 0,45$).

Chỉ số chất lượng tín hiệu (Signal Quality, SQ) nằm trong khoảng 0–100 và được tổng hợp từ hai thành phần: *độ đều R–R* (R-R regularity, trọng số 70%) — đo qua hệ số biến thiên (CV) của các khoảng R–R, và *độ phủ nhịp* (beat coverage, trọng số 30%) — tỷ lệ số nhịp phát hiện được so với mức kỳ vọng (khoảng 6 nhịp trong cửa sổ 6 giây). Khi mất hoàn toàn tín hiệu fECG, chỉ số SQ suy giảm theo hệ số 0,5 mỗi chu kỳ.

0.5.3 Cảnh báo lâm sàng

Hệ thống cảnh báo sử dụng ngưỡng nhịp tim thai theo khuyến cáo NICHD/ACOG và FIGO cho thai kỳ đơn thai, đủ tháng. Nhịp tim thai bình thường nằm trong khoảng **110–160 bpm**. Các trạng thái cảnh báo bao gồm:

- **Chậm nhịp (bradycardia):** FHR < 110 bpm — mã cảnh báo `low`.
- **Nhanh nhịp (tachycardia):** FHR > 160 bpm — mã cảnh báo `high`.
- **Mất tín hiệu (signal loss):** chỉ số chất lượng SQ < 30 hoặc không thể ước lượng FHR — mã cảnh báo `signal`.
- **Bình thường:** FHR trong khoảng 110–160 bpm và SQ ≥ 30 — mã trạng thái `ok`.

0.5.4 Lưu trữ và mô hình dữ liệu

Dữ liệu được lưu trữ trong cơ sở dữ liệu SQLite ở chế độ WAL (Write-Ahead Logging) để tối ưu hiệu năng ghi đồng thời. Nhằm tiết kiệm dung lượng lưu trữ, hệ thống chỉ ghi lại 1 trong mỗi 4 mẫu (downsample tỷ lệ 1:4) vào cơ sở dữ liệu, trong khi toàn bộ mẫu vẫn được phân phối live qua WebSocket.

Lược đồ cơ sở dữ liệu gồm năm bảng chính:

- `doctors`: thông tin bác sĩ (`id`, `username`, `password_hash`, `name`, `created_at`).
- `patients`: thông tin bệnh nhân (`id`, `name`, `mrn`, `sex`, `dob`, `notes`, `created_at`, `archived`).
- `sessions`: phiên thu nhận (`id`, `patient_id`, `started_at`, `ended_at`, `sample_rate`).
- `samples`: dữ liệu mẫu đã downsample (`id`, `session_id`, `t`, `raw`, `fecg`).
- `doctor_login`: phiên đăng nhập web (`token`, `doctor_id`, `created_at`).

Mật khẩu bác sĩ được mã hóa bằng thuật toán PBKDF2-HMAC-SHA256 với salt ngẫu nhiên 8 byte và 100.000 vòng lặp. Trường `archived` cho phép lưu trữ (ẩn) bệnh nhân không còn theo dõi; khi thiết bị kết nối lại cho bệnh nhân đã lưu trữ, hệ thống tự động bỏ lưu trữ (un-archive).

0.6 Phân hệ giao diện web cho bác sĩ

Giao diện web được thiết kế tối ưu cho màn hình cảm ứng 480×320 của Pi nhưng vẫn co giãn tốt trên các kích thước màn hình khác. Hệ thống gồm ba màn hình chính:

Màn hình đăng nhập (`/login`) trình bày một thẻ form ở giữa màn hình với các ô nhập liệu kích thước lớn phù hợp thao tác cảm ứng, nút “Sign in” chiếm toàn bộ chiều rộng, thông báo lỗi inline và hỗ trợ phím Enter. Phiên đăng nhập được quản lý bằng cookie `httponly` với `samesite=lax`.

Bảng điều khiển (`dashboard`, `/`) hiển thị danh sách bệnh nhân dưới dạng các thẻ cảm ứng. Mỗi thẻ bệnh nhân bao gồm: chấm trạng thái (`live/idle`), tên bệnh nhân, mã ID/MRN, nhịp tim mẹ (MHR), đường sparkline fECG thu nhỏ, và con số FHR lớn nổi bật. Bệnh nhân đang có cảnh báo bất thường được tự động sắp xếp lên đầu danh sách và tô nền đỏ (FHR ngoài khoảng bình thường) hoặc hiển thị biểu tượng cảnh báo (mất tín hiệu). Danh sách tự động làm mới mỗi 2 giây. Hệ thống hỗ trợ tìm kiếm bệnh nhân theo tên, ID hoặc MRN, cùng nút “+” để thêm bệnh nhân mới thông qua biểu mẫu dạng bottom sheet.

Màn hình bệnh nhân (`/patient/{id}`) chia thành hai tab:

- Tab *Monitor*: hiển thị ô FHR và MHR kích thước lớn với nhãn “normal 110–160”, thanh hiển thị chất lượng tín hiệu, banner cảnh báo (`bradycardia/tachycardia/signal loss`), hai đường tín hiệu cuộn (`raw` — màu cyan, `fECG` — màu hồng) có lưới chia và tự động co giãn trục Y, cùng nút “freeze” cho phép đóng băng dạng sóng để kiểm tra chi tiết.

- Tab *Info*: thông tin chi tiết bệnh nhân, thống kê số mẫu và số phiên đã ghi, bảng lịch sử phiên, và các thao tác Edit/Archive.

Dữ liệu live được cập nhật qua kênh WebSocket riêng cho từng bệnh nhân (`/ws/live/{id}`), với cơ chế tự động kết nối lại khi mất kết nối và gửi ping mỗi 25 giây để duy trì socket.

0.7 Kiểm thử và đánh giá hệ thống

Hệ thống đã được kiểm thử end-to-end theo kịch bản: thiết bị Pi (hoặc `stream_sim.py` chạy từ máy bất kỳ) phát dữ liệu qua WebSocket tới server, server xử lý phân tích và phân phối tới dashboard, dashboard hiển thị tín hiệu cuộn và các chỉ số lâm sàng live.

Các hạng mục kiểm thử đã thực hiện trong môi trường sandbox bao gồm:

- Kiểm thử xác thực: gating quyền truy cập (mã trả về 401 khi chưa đăng nhập, 200 khi đã xác thực), handshake thiết bị thành công và từ chối token sai.
- Kiểm thử luồng dữ liệu: broadcast WebSocket tới dashboard subscriber hoạt động đúng, phép toán lưu mẫu (downsample 1:4) chính xác.
- Kiểm thử bộ xử lý thời gian thực: `FecgProcessor` cho đầu ra fECG mượt mà qua cơ chế ring buffer + replay.
- Kiểm thử đường dự phòng: chế độ simulator và placeholder filter hoạt động đúng khi không có phần cứng/mô hình.
- Xác thực phân tích lâm sàng: tín hiệu tổng hợp với nhịp mẹ 75 bpm và nhịp thai 140 bpm cho kết quả $FHR \approx 140$, $MHR \approx 75$, $SQ \approx 97$, `alarm = ok` — khớp với kỳ vọng.

Trong các bước tiếp theo, hệ thống cần được bổ sung kiểm thử: đo độ trễ chuỗi end-to-end (từ ADS1293 đến hiển thị trên dashboard), đánh giá độ ổn định vận hành dài hạn (chạy liên tục nhiều giờ), và kiểm thử kịch bản ngắt/khôi phục kết nối mạng.

0.8 Kết luận chương

Chương này đã trình bày việc thiết kế và hiện thực thành công một hệ thống theo dõi fECG ba tầng hoạt động theo thời gian thực trên thiết bị biên Raspberry Pi 5. Hệ thống bao gồm đầy đủ các phân hệ từ thu nhận tín hiệu phần cứng (ADS1293 qua SPI phần mềm), xử lý và suy luận AI tại chỗ (ONNX INT8 với bộ đệm vòng),

truyền dữ liệu qua WebSocket, phân tích lâm sàng phía server (FHR/MHR/SQ/cảnh báo theo ngưỡng NICHD/ACOG), lưu trữ SQLite, cho đến giao diện dashboard web tối ưu cho cảm ứng. Kết quả này hiện thực hóa hai đóng góp chính đã nêu ở Chương ??: tối ưu và triển khai mô hình trên thiết bị biên (đóng góp số 2) và xây dựng hệ thống theo dõi lâm sàng hoàn chỉnh (đóng góp số 3).